

小组名称：新建队伍名

小组成员：董俊宏

报名赛题编号：T3-1-1

拟提交的 .skill 名称：nt-devskill

一、摘要

本项目旨在为九齿(NineToothed)框架构建一个面向 AI 智能体(如 Claude Code、Codex 等)的自动化开发 Skill，使智能体能够理解九齿框架的核心编程范式，并自动完成算子开发、正确性验证、性能测试和报告生成等任务。该项目将参加“2026 春季人工智能大赛·九齿 .skill 创新挑战赛道”。

九齿(NineToothed)是一个基于 Triton 的领域特定语言(DSL)，通过引入面向张量的元编程(Tensor-Oriented Meta-Programming, TOM)，使编写高性能 GPU 内核变得更容易。

本 Skill 将充分挖掘九齿框架串行语义自动转并行的核心优势，以“arrange-and-apply”范式为核心指导，赋能 AI 智能体成为九齿算子开发的专家助手。

二、项目背景

2.1 九齿框架概述

九齿是一个基于 Triton 的领域特定语言，其核心创新在于面向张量的元编程(TOM)和“arrange-and-apply”编程范式——将张量的布局编排(arrangement)与计算逻辑(application)分离定义，再通过 ninetoothed.make 集成生成内核。这种设计使开发者能够以串行语义编写并行程序，大幅简化 GPU 内核开发流程，同时保持与 Triton 相当的性能。

2.2 目前研究现状

ESP-DL 框架的 espdl-operator skill,能够指导 AI 自动完成算子需求分析、代码生成、测试用例创建、构建执行和结果验证的全流程。

九齿官方提供了丰富的参考资料：包括 GitHub 主仓库、ntops 算子库、examples 示例库，以及 arXiv 论文《NineToothed: A Triton-Based High-Level

Domain-Specific Language for Machine Learning》。

三、 Skill 项目目标

本方案目标是构建一个高效率、高可靠、可复用的 AI Agent skill 项目，用于指导 AI 智能体完成九齿（NineToothed）生态中的算子开发全流程，包括从需求理解到可运行且有性能意识的实现，形成闭环。

3.1 核心目标

本 Skill 的核心价值，是将算子开发经验从“隐性知识”工程化为“可调用、可组合、可复用的 AI 可执行能力”。

以 NineToothed 为代表的 DSL 已为开发者提供了高层次的编程抽象，但 AI 智能体仍需要系统性的方法论指导，才能稳定产出高质量算子。本 Skill 的价值便在于：

- 填补“需求”与“实现”的鸿沟：教 AI 如何从算子数学定义出发，选择合适的 NineToothed DSL 表达方式。
- 构建“开发 → 验证 → 优化 → 修复”闭环：不是一次性代码生成，而是完成工程迭代式的质量保证。
- 沉淀最佳实践：将 NineToothed 生态的 common patterns，官方示例等编码为可被自然语言轻松复用的资源，降低开发者上手 NineToothed 的难度。

本 Skill 安装后，AI 智能体能够稳定实现以下能力：

表 1 Skill 核心目标

能力维度	具体内容	交付指标
算子编写	能够理解自然语言或者高级编程语言表示的数学定义，使用 NineToothed DSL 构造 arrangement + application	算子正确性、可编译性
正确性验证	能够生成待测试算子 correctness 测试用例并执行验证	测试通过率
性能优化	设计 benchmark、分析瓶颈、调优配置	性能测试结果
调试与修复	能够根据错误信息，定位失败原因，给出最小修复，并完成验证闭环	修复成功率、迭代效率
集成与输出	符合仓库风格、产出文档/示例	代码质量、可复现性

3.2 覆盖算子类型

参考九齿 `examples` 和 `ntops` 项目，本 Skill 将覆盖以下算子类别：

表 2 Skill 覆盖算子类型

类别	代表算子	来源参考
基础运算	向量加法 (Vector Add)、矩	ninetoothed-examples
	阵乘法 (MatMul)	
激活函数	ReLU、Sigmoid、GELU	ntops
规约操作	Softmax、LayerNorm	ninetoothed-examples
卷积与池化	Conv2D、MaxPool2D	ninetoothed-examples / ntops
注意力机制	Flash Attention 核心组件	ntops / ninetoothed-examples
LLM 专用算子	RMSNorm、Rotary	ntops
	Embedding、SwiGLU	

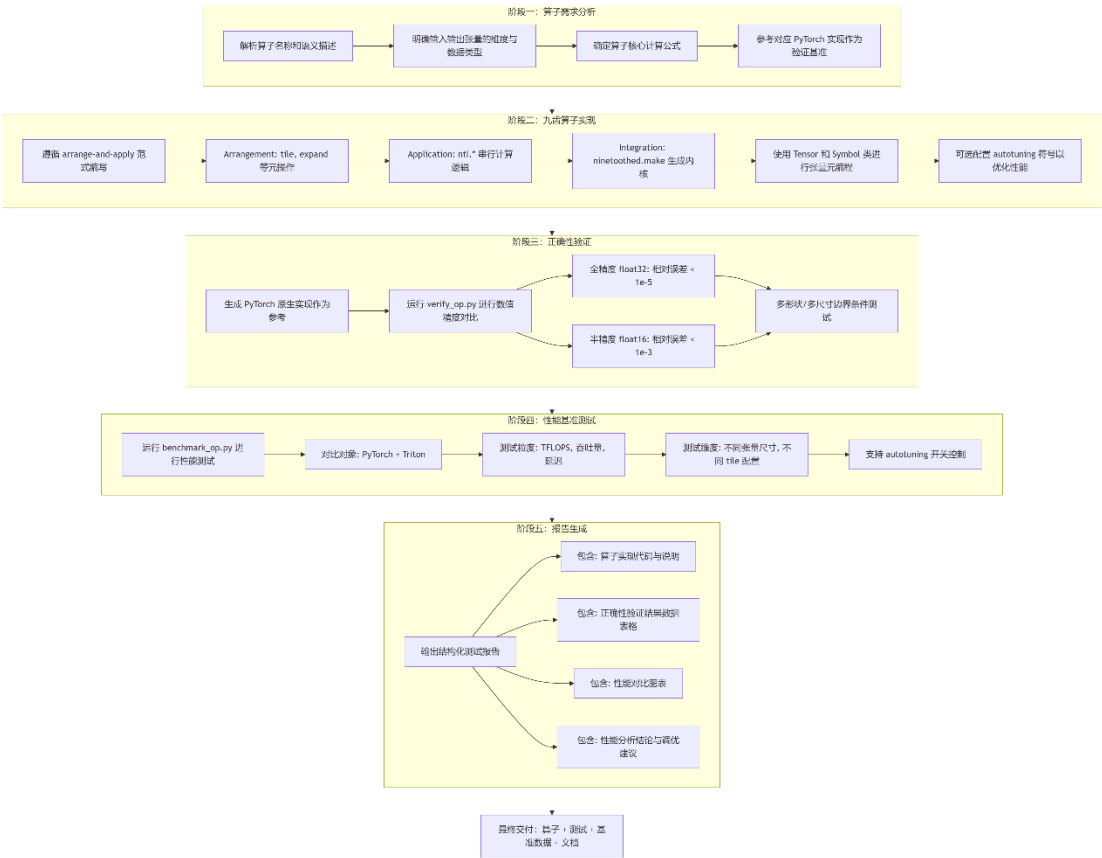
3.3 评测能力基线

本 Skill 的目标是在标准评测任务中，AI 智能体能够独立完成指定算子的端到端开发，正确性验证通过率达到 90% 以上，性能测试结果相对于 Triton 参考实现的性能损失控制在 15% 以内。

四、 方案设计

4.1 Skill 整体架构

Skill 将按照以下层级处理请求：



Skill 目录结构如下：

```
ninetoothed-skill/
├── SKILL.md                # 核心技能文档：触发条件、工作流、约束与验证方式
├── references/
│   ├── api_reference.md    # NineToothed API 完整参考（Tensor、Symbol、ntl 等）
│   ├── arrange_patterns.md # arrangement 编排模式库与示例
│   ├── optimization_guide.md # 性能调优指南（tile 尺寸选择、autotuning 配置等）
│   └── troubleshooting.md # 常见问题排查与调试技巧
├── scripts/
│   ├── verify_op.py        # 算子正确性验证脚本（数值比对、精度分析）
│   ├── benchmark_op.py     # 性能基准测试脚本（支持 Triton 对照）
│   ├── gen_report.py       # 测试报告生成器
│   └── setup_env.sh        # 环境初始化脚本（pip install ninetoothed torch 等）
├── examples/
│   ├── vector_add/         # 完整示例：向量加法
│   │   ├── requirement.md  # 算子需求说明
│   │   ├── implementation.py # 九齿实现代码
│   │   └── test_report.md  # 测试报告样例
│   └── matmul/             # 完整示例：矩阵乘法
├── tests/
│   ├── test_correctness.py # 正确性测试套件
│   ├── test_benchmark.py   # 性能测试套件
│   └── test_skill_validity.py # Skill 自身有效性测试
└── README.md              # 安装与使用说明
```

4.2 SKILL.md 核心设计

SKILL.md 将遵循以下结构：

4.2.1 触发场景定义

Skill 应在以下场景被激活：

- 用户请求“用九齿实现一个 [算子名称] 算子”
- 用户请求“优化 [算子名称] 的九齿实现”
- 用户请求“对九齿算子进行性能测试/基准测试”
- 用户请求“调试/验证九齿算子的正确性”

4.2.2 标准工作流

SKILL.md 将定义统一的标准工作流，包含 5 个核心阶段：

- 阶段一：算子需求分析
 - 解析算子名称和语义描述
 - 明确输入输出张量的维度与数据类型
 - 确定算子核心计算公式
 - 参考对应 PyTorch 实现作为验证基准
- 阶段二：九齿算子实现
 - 遵循 arrange-and-apply 范式编写：
 - Arrangement 阶段：使用 `tile()`、`expand()` 等元操作完成张量布局编排
 - Application 阶段：使用 `ntl.*`（如 `ntl.dot()`、`ntl.zeros()`）编写串行计算逻辑
 - Integration 阶段：通过 `ninetoothed.make()` 生成最终内核
 - 使用 `Tensor` 和 `Symbol` 类进行张量元编程
 - 可选配置 `autotuning` 符号以优化性能
- 阶段三：正确性验证
 - 生成 PyTorch 原生实现作为参考
 - 运行 `verify_op.py` 进行数值精度对比：
 - 全精度（`float32`）误差测试：相对误差 $< 1e-5$
 - 半精度（`float16`）误差测试：相对误差 $< 1e-3$
 - 多形状/多尺寸边界条件测试
- 阶段四：性能基准测试

- 运行 `benchmark_op.py` 进行性能测试：
- 对比对象：PyTorch 原生实现 + Triton 实现（若可用）
- 测试粒度：TFLOPS、吞吐量（ops/s）、延迟（ms）
- 测试维度：不同张量尺寸、不同 tile 配置
- 支持 autotuning 开关控制
- 阶段五：报告生成
 - 输出结构化测试报告，包含：
 - 算子实现代码与说明
 - 正确性验证结果（数据表格）
 - 性能对比图表（吞吐量 vs 尺寸）
 - 性能分析结论与调优建议

4.3 关键技术设计

4.3.1 算子模板系统

基于九齿框架的 `arrange-and-apply` 范式，设计可复用的算子代码模板，如：

```
# 基本模板结构
from ninetoothed import Tensor, block_size, make
import ninetoothed.language as ntl

BLOCK_SIZE_M = block_size()
BLOCK_SIZE_N = block_size()
BLOCK_SIZE_K = block_size()

def arrangement(input, other, output):
    # 使用 tile, expand 等元操作
    ...

def application(input, other, output):
    # 使用 ntl 编写串行计算
    ...

tensors = (Tensor(2), Tensor(2), Tensor(2))
kernel = make(arrangement, application, tensors)
```

针对不同算子类型（逐元素操作、规约操作、矩阵乘法、卷积）提供差异化的模板变体。

4.3.2 验证框架设计

验证框架将实现以下核心功能：

- 动态对比验证：自动生成测试数据，同时运行九齿内核和 PyTorch 参考

实现，计算最大/平均/相对误差。

- 边界条件覆盖: 自动测试多种张量形状(包括非对齐尺寸、极端尺寸等)。
- 精度诊断: 当误差超阈值时，自动输出误差分布热力图和位置定位。

4.3.3 性能测试框架设计

性能测试框架将实现：

- 自动规模扫描: 生成多种张量尺寸组合（如 128→4096 步进）。
- 对照实验: 与 PyTorch 原生实现、Triton 实现（如有）进行性能对比。
- 数据可视化: 生成性能对比图表（吞吐量 vs 尺寸的折线图）。
- 最优配置推荐: 根据测试结果推荐最优的 tile 尺寸和 autotuning 配置。

五、项目实施计划

5.1 时间安排

表 3 时间安排

阶段	时间	主要工作
Phase 1: 基础框架搭建	第 1-3 天	完成 SKILL.md 框架编写，实现环境配置脚本
Phase 2: 核心脚本开发	第 4-7 天	完成 verify_op.py、benchmark_op.py 等核心脚本
Phase 3: 参考资料整理	第 8-10 天	整理 API 参考文档、模式库、优化指南
Phase 4: 示例编写	第 11-14 天	完成 vector_add 和 matmul 两个完整示例
Phase 5: 测试与优化	第 15-18 天	完成 tests/ 目录下的测试套件，进行 Skill 有效性验证
Phase 6: 报告与提交	第 19-21 天	生成项目报告，准备最终提交材料

5.2 里程碑检查点

M1（第 3 天）：SKILL.md 完成，触发场景和工作流定义清晰

M2（第 7 天）：核心脚本可独立运行，能够完成正确的验证和测试

M3（第 14 天）：至少 2 个完整示例可供 AI 智能体学习和执行

M4（第 18 天）：测试套件全部通过，Skill 有效性得到验证

M5（第 21 天）：项目报告完成，符合赛道提交规范

六、预期成果

本项目将交付以下成果物：

1. 完整的 .skill 包：按照赛道规定结构组织，包含 SKILL.md、references/、scripts/、examples/、tests/ 和 README.md，可在指定环境中离线复现
2. 可参考的算子示例库：至少 2 个完整的算子开发与验证示例，可作为 AI 智能体的学习蓝本
3. 项目报告：详细说明 Skill 的设计思路、实现细节、评测方式和验证结果